

Informe Laboratorio 5

Sección 1

Bruno Figueroa
e-mail: bruno.figueroa@mail.udp.cl

Noviembre de 2025

Índice

Descripción de actividades	4
1. Desarrollo (Parte 1)	7
1.1. Códigos de cada Dockerfile	7
1.1.1. C1	7
1.1.2. C2	8
1.1.3. C3	9
1.1.4. C4/S1	10
1.2. Creación de las credenciales para S1	11
1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	12
1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	13
1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	13
1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)	14
1.7. Compara la versión de HASSH obtenida con la base de datos para validar si el cliente corresponde al mismo	15
1.8. Tipo de información contenida en cada uno de los paquetes generados en texto plano	15
1.8.1. C1	15
1.8.2. C2	15
1.8.3. C3	16
1.8.4. C4/S1	16
1.9. Diferencia entre C1 y C2	16
1.10. Diferencia entre C2 y C3	16
1.11. Diferencia entre C3 y C4	16
2. Desarrollo (Parte 2)	17
2.1. Identificación del cliente SSH con versión “?”	17
2.2. Replicación de tráfico al servidor (paso por paso)	17
2.2.1. Preparación del entorno	17
2.2.2. Inicio de captura de paquetes	17
2.2.3. Generación del tráfico hacia el servidor	17
2.2.4. Finalización de la captura	17
2.2.5. Extracción del archivo pcap generado	18
2.2.6. Análisis del flujo replicado	18
2.2.7. Repetición para C2, C3 y C4/S1	18

3. Desarrollo (Parte 3)	18
3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)	18
4. Desarrollo (Parte 4)	18
4.1. Explicación OpenSSH en general	18
4.2. Capas de Seguridad en OpenSSH	19
4.3. Identificación de que protocolos no se cumplen	20

Descripción de actividades

Para este último laboratorio, se solicita trabajar con Docker y el protocolo SSH, a fin de poder entender el concepto de criptografía asimétrica y firmas digitales.

Para lo anterior deberá:

- Crear 4 contenedores en Docker por medio de un DockerFile, donde cada uno tendrá el siguiente SO: Ubuntu 16.10, Ubuntu 18.10, Ubuntu 20.10 y Ubuntu 22.10 a los cuales se llamarán C1, C2, C3 y C4 respectivamente.
El equipo con Ubuntu 22.10 también será utilizado como S1.
- Para cada uno de ellos, deberá instalar el cliente openSSH disponible en los repositorios de apt, y para el equipo S1 deberá también instalar el servidor openSSH.
- En S1 deberá crear el usuario “**prueba**” con contraseña “**prueba**”, para acceder a él desde los clientes por el protocolo SSH.
- En total serán 4 escenarios, donde cada uno corresponderá a los siguientes equipos:
 - C1 → S1
 - C2 → S1
 - C3 → S1
 - C4 → S1

Pasos:

1. Para cada uno de los 4 escenarios, solo deberá establecer la conexión y no realizar ningún otro comando que pueda generar tráfico (como muestra la Figura). Deberá capturar el tráfico de red generado y analizar el patrón de tráfico generado por cada cliente. De esta forma podrá obtener una huella digital para cada cliente a partir de su tráfico.

Indique el tamaño de los paquetes del flujo generados por el cliente y el contenido asociado a cada uno de ellos. Indique qué información distinta contiene el escenario siguiente (diff incremental). El objetivo de este paso es identificar claramente los cambios entre las distintas versiones de ssh.

2. Para poder identificar que el usuario efectivamente es el informante, éste utilizará una versión única de cliente. ¿Con qué cliente SSH se habrá generado el siguiente tráfico?

Protocol	Length	Info
TCP	74	34328 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=14
TCP	66	34328 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0
SSHv2	85	Client: Protocol (SSH-2.0-OpenSSH_?)
TCP	66	34328 → 22 [ACK] Seq=20 Ack=42 Win=64256 Len=
SSHv2	1578	Client: Key Exchange Init
TCP	66	34328 → 22 [ACK] Seq=1532 Ack=1122 Win=64128
SSHv2	114	Client: Elliptic Curve Diffie-Hellman Key Exc
TCP	66	34328 → 22 [ACK] Seq=1580 Ack=1574 Win=64128
SSHv2	82	Client: New Keys
SSHv2	110	Client: Encrypted packet (len=44)
TCP	66	34328 → 22 [ACK] Seq=1640 Ack=1618 Win=64128
SSHv2	126	Client: Encrypted packet (len=60)
TCP	66	34328 → 22 [ACK] Seq=1700 Ack=1670 Win=64128
SSHv2	150	Client: Encrypted packet (len=84)
TCP	66	34328 → 22 [ACK] Seq=1784 Ack=1698 Win=64128
SSHv2	178	Client: Encrypted packet (len=112)
TCP	66	34328 → 22 [ACK] Seq=1896 Ack=2198 Win=64128

Figura 1: Tráfico generado del informante

Replique este tráfico generado en la imagen. Debe generar el tráfico con la misma versión resaltada en azul. Recuerde que toda la información generada es parte del sw, por lo tanto usted puede modificar toda la información.

3. Para que el informante esté seguro de nuestra identidad, nos pide que el patrón del tráfico de nuestro server también sea modificado, hasta que el Key Exchange Init del server sea menor a 300 bytes. Indique qué pasos realizó para lograr esto.

TCP	66	42350 → 22	[ACK]	Seq=2	Ack=
TCP	74	42398 → 22	[SYN]	Seq=0	Win=
TCP	74	22 → 42398	[SYN, ACK]	Seq=0	
TCP	66	42398 → 22	[ACK]	Seq=1	Ack=
SSHv2	87	Client: Protocol (SSH-2.0-C			
TCP	66	22 → 42398	[ACK]	Seq=1	Ack=
SSHv2	107	Server: Protocol (SSH-2.0-C			
TCP	66	42398 → 22	[ACK]	Seq=22	Ack=
SSHv2	1570	Client: Key Exchange Init			
TCP	66	22 → 42398	[ACK]	Seq=42	Ack=
SSHv2	298	Server: Key Exchange Init			
TCP	66	42398 → 22	[ACK]	Seq=1526	A

Figura 2: Captura del Key Exchange

- Tomando en cuenta lo aprendido en este laboratorio, así como en los anteriores, explique el protocolo OpenSSH y las diferentes capas de seguridad que son parte del protocolo para garantizar los principios de seguridad de la información, integridad, confidencialidad, disponibilidad, autenticidad y no repudio. Es importante que sea muy específico en el objetivo del principio en el protocolo. En caso de considerar que alguno de los principios no se cumple, justifique su razonamiento. Es fundamental que su análisis se base en el tráfico SSH interceptado.

1. Desarrollo (Parte 1)

1.1. Códigos de cada Dockerfile

Utilizando tecnologia docker compose, se despliegan los siguientes 4 contenedores C1, C2, C3 y C4, el cual tambien actua como S1, se muestra a continuacion el codigo respectivo a cada uno, junto con una descripcion de estos.

1.1.1. C1

Se muestra en la figura 3 el codigo para el cliente 1, donde se utiliza una version de ubuntu 16.10, en la cual se instala openssh-client y otras dependencias necesarias para su funcionamiento, al final del archivo se muestra un comando cmd, sleep infinity, que se utiliza para que el contenedor no termine su ejecucion apenas sea creado, y en caso de estar inactivo, se ponga a dormir, esperando un comando del usuario.

```
# Dockerfile C1
ARG UBUNTU_VERSION=16.10
FROM ubuntu:${UBUNTU_VERSION}

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        openssh-client \
        ca-certificates \
        iproute2 \
        iputils-ping \
        net-tools \
        tcpdump \
        vim \
        less \
        busybox \
        sshpass || true

CMD ["sleep", "infinity"]
```

Figura 3: Archivo DockerFile C1

1.1.2. C2

Se muestra en la figura 4 el código para el cliente 2, donde se utiliza una versión de ubuntu 18.10, en la cual se instala openssh-client y otras dependencias necesarias para su funcionamiento, al final del archivo se muestra un comando cmd, sleep infinity, que se utiliza para que el contenedor no termine su ejecución apenas sea creado, y en caso de estar inactivo, se ponga a dormir, esperando un comando del usuario.

```
# Dockerfile C2
ARG UBUNTU_VERSION=18.10
FROM ubuntu:${UBUNTU_VERSION}

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        openssh-client \
        ca-certificates \
        iproute2 \
        iputils-ping \
        net-tools \
        tcpdump \
        vim \
        less \
        busybox \
        sshpass || true

CMD ["sleep", "infinity"]
```

Figura 4: Archivo DockerFile C2

1.1.3. C3

Se muestra en la figura 5 el código para el cliente 3, donde se utiliza una versión de ubuntu 20.10, en la cual se instala openssh-client y otras dependencias necesarias para su funcionamiento, al final del archivo se muestra un comando cmd, sleep infinity, que se utiliza para que el contenedor no termine su ejecución apenas sea creado, y en caso de estar inactivo, se ponga a dormir, esperando un comando del usuario.

```
# Dockerfile C3
ARG UBUNTU_VERSION=20.10
FROM ubuntu:${UBUNTU_VERSION}

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        openssh-client \
        ca-certificates \
        iproute2 \
        iputils-ping \
        net-tools \
        tcpdump \
        vim \
        less \
        busybox \
        sshpass || true

CMD ["sleep", "infinity"]
```

Figura 5: Archivo DockerFile C3

1.1.4. C4/S1

Finalmente, se muestra en la figura 6 el código para el cliente 2, donde se utiliza una versión de ubuntu 22.10, en la cual se instala openssh-client, openssh-server y otras dependencias necesarias para su funcionamiento, luego, se tiene un script de arranque para el servidor "start-ssh.sh", el cual realiza la configuración inicial, como la comprobación del funcionamiento de PasswordAuthenticator, el cual se utilizara mas adelante.

```

# Dockerfile C4/S1
ARG UBUNTU_VERSION=22.10
FROM ubuntu:${UBUNTU_VERSION}

ENV DEBIAN_FRONTEND=noninteractive

# Repositorios old-releases para kinetic (EOL)
RUN sed -i 's|http://archive.ubuntu.com/ubuntu|http://old-releases.ubuntu.com/ubuntu|g' /etc/apt/sources.list \
&& sed -i 's|http://security.ubuntu.com/ubuntu|http://old-releases.ubuntu.com/ubuntu|g' /etc/apt/sources.list

RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        openssh-server \
        pwgen \
        openssh-client \
        ca-certificates \
        iproute2 \
        iputils-ping \
        net-tools \
        tcpdump \
        vim \
        less \
        busybox && \
    mkdir -p /var/run/sshd

# Script de arranque
COPY start-sshd.sh /usr/local/bin/start-sshd.sh
RUN chmod +x /usr/local/bin/start-sshd.sh

CMD ["/usr/local/bin/start-sshd.sh"]

```

Figura 6: Archivo DockerFile C4

1.2. Creación de las credenciales para S1

En la figura 7, se muestra el proceso utilizado para crear la credencial para el servidor, en la cual se ejecuta el comando primer comando, para crear un usuario llamado prueba, con la contraseña prueba, esto no retorna nada, ya que solo esta creando un archivo.

Luego, se ejecuta el comando docker exec -it C4 id prueba, entregando uid=1000(prueba) gid=1000(prueba) groups=1000(prueba) demostrando que el archivo fue creado exitosamente dentro del contenedor.

Y finalmente se utiliza el ultimo comando, el cual testea si PasswordAuthentication esta activo y funcionando, y en este caso, retorno "PasswordAuthentication yes" lo que significa que esta activo y funcionando sin problemas.

```

ndesktop@MDesktop:~$ docker exec -it C4 bash -c "useradd -m -s /bin/bash prueba || true; echo 'prueba:prueba' | chpasswd"
ndesktop@MDesktop:~$ docker exec -it C4 id prueba
uid=1000(prueba) gid=1000(prueba) groups=1000(prueba)
ndesktop@MDesktop:~$ docker exec -it C4 bash -c "grep -i '^PasswordAuthentication' /etc/ssh/sshd_config || echo 'PasswordAuthentication yes' >> /etc/ssh/sshd_config"
PasswordAuthentication yes
ndesktop@MDesktop:~$

```

Figura 7: Comandos para la creacion de credenciales

1.3. Tráfico generado por C1, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Ahora, para generar el trafico necesario, se utilizaron los comandos que se muestran en la figura 8, donde, comienza la captura de paquetes, luego envia un mensaje ssh para conectarse, utilizando la contraseña "prueba" y finalmente dejando como output un archivo pcap con todos los paquetes obtenidos.

```
mdesktop@MDesktop:~$ docker exec -d C1 bash -c "tcpdump -i eth0 -s 0 -w /tmp/c1.pcap tcp and host C4"
docker exec -it C1 ssh -oStrictHostKeyChecking=no prueba@C4 true
docker cp C1:/tmp/c1.pcap ./c1.pcap
Warning: Permanently added 'c4,172.18.0.2' (ECDSA) to the list of known hosts.
prueba@c4's password:
Successfully copied 5.63kB to /home/mdesktop/c1.pcap
```

Figura 8: Comandos para conexion entre C1 y S1

Luego, en la figura 9, se muestra el pcap generado, en el cual se observa los siguientes paquetes creados por el cliente:

1. Frame 1 (TCP, 74) - SYN (control TCP).
2. Frame 3 (TCP, 66) - ACK (control TCP).
3. Frame 6 (SSHv2, 107) - Client banner: ASCII. Visible en claro; identifica versión del cliente.
4. Frame 8 (SSHv2, 1498) - Client: $SSH_MSG_KEXINIT$: *Contenidoentexto/bytes : listasASCIIdealgoritmos.Esteeselmayorelementodehuellaconlongitudde1498bytes.*
4. Frame 11 (SSHv2, 114) - Client: ECDH Key Exchange Init (clave publica ECDH, binario). Contenido binario no legible en ASCII, tamaño de 114 bytes.

Estos paquetes se pueden interpretar de la siguiente manera: El cliente C1 identifica como OpenSSH_{7,3p1}yenviaunKEXINITde1498bytescomohuellacaracteristica

1	0.000000	172.18.0.4	172.18.0.2	TCP	74	53182 → 22 [SYN] Seq=0
2	0.000017	172.18.0.2	172.18.0.4	TCP	74	22 → 53182 [SYN, ACK] S
3	0.000026	172.18.0.4	172.18.0.2	TCP	66	53182 → 22 [ACK] Seq=1
4	0.000770	172.18.0.2	172.18.0.4	SSHv2	107	Server: Protocol (SSH-2
5	0.000777	172.18.0.4	172.18.0.2	TCP	66	53182 → 22 [ACK] Seq=1
6	0.002634	172.18.0.4	172.18.0.2	SSHv2	107	Client: Protocol (SSH-2
7	0.002647	172.18.0.2	172.18.0.4	TCP	66	22 → 53182 [ACK] Seq=42
8	0.002762	172.18.0.4	172.18.0.2	SSHv2	1498	Client: Key Exchange In
9	0.002772	172.18.0.2	172.18.0.4	TCP	66	22 → 53182 [ACK] Seq=42
10	0.003658	172.18.0.2	172.18.0.4	SSHv2	1146	Server: Key Exchange In
11	0.004744	172.18.0.4	172.18.0.2	SSHv2	114	Client: Elliptic Curve

Figura 9: Archivo pcap en conexion entre C1 y S1

1.4. Tráfico generado por C2, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Ahora, para generar el trafico necesario, se utilizaron los comandos que se muestran en la figura 10, donde, comienza la captura de paquetes, luego envia un mensaje ssh para conectarse, utilizando la contraseña "prueba" y finalmente dejando como output un archivo pcap con todos los paquetes obtenidos.

```
mdesktop@MDesktop:~$ docker exec -d C2 bash -c "tcpdump -i eth0 -s 0 -w /tmp/c2.pcap tcp and host C4"
docker exec -it C2 ssh -oStrictHostKeyChecking=no prueba@C4 true
docker cp C2:/tmp/c2.pcap ./c2.pcap
Warning: Permanently added 'c4,172.18.0.2' (ECDSA) to the list of known hosts.
prueba@c4's password:
Successfully copied 5.63kB to /home/mdesktop/c2.pcap
```

Figura 10: Comandos para conexion entre C2 y S1

Luego, en la figura 11, se muestra el pcap generado, en el cual se observa los siguientes paquetes creados por el cliente:

1. Client banner: frame 6, size 107 — OpenSSH_{7,7p1}.
1. Client KEXINIT: frame 8, size 1426 bytes.
2. Client ECDH init: frame 11, size 114.

Estos paquetes se pueden interpretar de la siguiente manera: El cliente C2 usa OpenSSH_{7,7p1}; *suK EX*

1	0.000000	172.18.0.5	172.18.0.2	TCP	74 55886 → 22 [SYN] Seq=0 W
2	0.000017	172.18.0.2	172.18.0.5	TCP	74 22 → 55886 [SYN, ACK] Seq
3	0.000026	172.18.0.5	172.18.0.2	TCP	66 55886 → 22 [ACK] Seq=1 Ac
4	0.000721	172.18.0.2	172.18.0.5	SSHv2	107 Server: Protocol (SSH-2.6
5	0.000731	172.18.0.5	172.18.0.2	TCP	66 55886 → 22 [ACK] Seq=1 Ac
6	0.002570	172.18.0.5	172.18.0.2	SSHv2	107 Client: Protocol (SSH-2.6
7	0.002582	172.18.0.2	172.18.0.5	TCP	66 22 → 55886 [ACK] Seq=42 A
8	0.002695	172.18.0.5	172.18.0.2	SSHv2	1426 Client: Key Exchange Init
9	0.002704	172.18.0.2	172.18.0.5	TCP	66 22 → 55886 [ACK] Seq=42 A
10	0.003592	172.18.0.2	172.18.0.5	SSHv2	1146 Server: Key Exchange Init
11	0.004675	172.18.0.5	172.18.0.2	SSHv2	114 Client: Elliptic Curve Di

Figura 11: Archivo pcap en conexion entre C2 y S1

1.5. Tráfico generado por C3, detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Ahora, para generar el trafico necesario, se utilizaron los comandos que se muestran en la figura 12, donde, comienza la captura de paquetes, luego envia un mensaje ssh para conectarse, utilizando la contraseña "prueba" y finalmente dejando como output un archivo pcap con todos los paquetes obtenidos.

Luego, en la figura 13, se muestra el pcap generado, en el cual se observa los siguientes paquetes creados por el cliente:

1.6 Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

1 DESARROLLO (PARTE 1)

```
mdesktop@MDesktop:~$ docker exec -d C3 bash -c "tcpdump -i eth0 -s 0 -w /tmp/c3.pcap tcp and host C4"
docker exec -it C3 ssh -oStrictHostKeyChecking=no prueba@C4 true
docker cp C3:/tmp/c3.pcap ./c3.pcap
Warning: Permanently added 'c4,172.18.0.2' (ECDSA) to the list of known hosts.
prueba@c4's password:
Successfully copied 5.63kB to /home/mdesktop/c3.pcap
```

Figura 12: Comandos para conexion entre C3 y S1

1. Client banner: frame 6, size 107 — OpenSSH_{8,3p1}.
1. Client KEXINIT: frame 8, size 1578 bytes (mayor que C1/C2).
2. Client ECDH: frame 11, size 114.

Estos paquetes se pueden interpretar de la siguiente manera: El cliente C3 utilizando OpenSSH_{8,3p1}, *envia el KEXINIT mas grande, con un largo de 1578, lo que sugiere que es un listado de algoritmos*

1	0.000000	172.18.0.3	172.18.0.2	TCP	74	58410 → 22 [SYN] Seq=0 Wi
2	0.000020	172.18.0.2	172.18.0.3	TCP	74	22 → 58410 [SYN, ACK] Seq
3	0.000031	172.18.0.3	172.18.0.2	TCP	66	58410 → 22 [ACK] Seq=1 Ac
4	0.000686	172.18.0.2	172.18.0.3	SSHv2	107	Server: Protocol (SSH-2.0
5	0.000693	172.18.0.3	172.18.0.2	TCP	66	58410 → 22 [ACK] Seq=1 Ac
6	0.002823	172.18.0.3	172.18.0.2	SSHv2	107	Client: Protocol (SSH-2.0
7	0.002836	172.18.0.2	172.18.0.3	TCP	66	22 → 58410 [ACK] Seq=42 A
8	0.002949	172.18.0.3	172.18.0.2	SSHv2	1578	Client: Key Exchange Init
9	0.002959	172.18.0.2	172.18.0.3	TCP	66	22 → 58410 [ACK] Seq=42 A
10	0.003795	172.18.0.2	172.18.0.3	SSHv2	1146	Server: Key Exchange Init
11	0.004816	172.18.0.3	172.18.0.2	SSHv2	114	Client: Elliptic Curve Di

Figura 13: Archivo pcap en conexion entre C3 y S1

1.6. Tráfico generado por C4 (iface lo), detallando tamaño paquetes del flujo y el HASSH respectivo (detallado)

Ahora, para generar el trafico necesario, se utilizaron los comandos que se muestran en la figura 14, donde, comienza la captura de paquetes, luego envia un mensaje ssh para conectarse, utilizando la contraseña "prueba" y finalmente dejando como output un archivo pcap con todos los paquetes obtenidos.

Luego, en la figura 15, se muestra el pcap generado, en el cual se observa los siguientes paquetes creados por el cliente:

1. Client banner: frame 4, size 107 — OpenSSH_{9,0p1}.
1. Client KEXINIT: frame 8, size 1570 bytes.
2. (No ECDH shown in extract; possibly later or merged.)

Estos paquetes se pueden interpretar de la siguiente manera: El server C4, cuando actua como cliente, utiliza OpenSSH_{9,0p1} *ysu KEXINIT es 1570 bytes, similar en tamaño a C3, siendo este de 1578*

1.7 Compara la versión de HASSH obtenida con la base de datos para validar si el cliente corresponde al mismo

1 DESARROLLO (PARTE 1)

```
mdesktop@MDesktop:~$ docker exec -d C4 bash -c "tcpdump -i lo -s 0 -w /tmp/c4_lo.pcap tcp port 22"
docker exec -it C4 ssh prueba@127.0.0.1 true
docker cp C4:/tmp/c4_lo.pcap ./c4_lo.pcap
The authenticity of host '127.0.0.1 (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:vLo7FV6dT+iUkVdVoa0749Ex8dXc05vpwfiGi0YLVBK.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '127.0.0.1' (ED25519) to the list of known hosts.
prueba@127.0.0.1's password:
Successfully copied 5.63kB to /home/mdesktop/c4_lo.pcap
```

Figura 14: Comandos para conexion entre C4 y S1

1	0.000000	127.0.0.1	127.0.0.1	TCP	74 54750 → 22 [SYN] Seq=0 Win=
2	0.000012	127.0.0.1	127.0.0.1	TCP	74 22 → 54750 [SYN, ACK] Seq=6
3	0.000022	127.0.0.1	127.0.0.1	TCP	66 54750 → 22 [ACK] Seq=1 Ack=
4	0.000192	127.0.0.1	127.0.0.1	SSHv2	107 Client: Protocol (SSH-2.0-C
5	0.000197	127.0.0.1	127.0.0.1	TCP	66 22 → 54750 [ACK] Seq=1 Ack=
6	0.000669	127.0.0.1	127.0.0.1	SSHv2	107 Server: Protocol (SSH-2.0-C
7	0.000677	127.0.0.1	127.0.0.1	TCP	66 54750 → 22 [ACK] Seq=42 Ack=
8	0.000783	127.0.0.1	127.0.0.1	SSHv2	1570 Client: Key Exchange Init
9	0.001580	127.0.0.1	127.0.0.1	SSHv2	1146 Server: Key Exchange Init
10	0.041603	127.0.0.1	127.0.0.1	TCP	66 54750 → 22 [ACK] Seq=1546 A

Figura 15: Archivo pcap en conexion entre C4 y S1

1.7. Compara la versión de HASSH obtenida con la base de datos para validar si el cliente corresponde al mismo

1.8. Tipo de información contenida en cada uno de los paquetes generados en texto plano

1.8.1. C1

El flujo inicia con el Handshake TCP: SYN, SYN/ACK y ACK. Luego el servidor envia su banner de protocolo SSH-2.0-OpenSSH_{9,0p1Ubuntu-1ubuntu7,3}.

El cliente responde con su banner SSH-2.0-OpenSSH_{7,3p1Ubuntu-1ubuntu0,1}.

A continuacion, el cliente envia el Key Exchange Init, que contiene los algoritmos que soporta para cifrado, MAC y compresion. El servidor responde con su propio Key Exchange Init.

Luego, sigue el Elliptic Curve Diffie-Hellman Key Exchange, el intercambio de claves temporales y la negociacion de nuevas llaves. Los paquetes posteriores contienen mensajes cifrados iniciales y confirmaciones para establecer la sesion.

1.8.2. C2

El flujo TCP es similar: SYN, SYN/ACK, ACK.

El servidor envia el mismo banner SSH-2.0-OpenSSH_{9,0p1Ubuntu-1ubuntu7,3}.

El vliente en este caso usa SSH-2.0-OpenSSH_{7,7p1Ubuntu-4ubuntu0,3,porloueelKeyexchangeInittio}

El resto del flujo sigue con ECDH Key exchange, para la negociacion de las llaves y la confirmacion de las iniciales cifradas.

1.8.3. C3

Al igual que en C1 y C2, handshake TCP inicial. Servidor envía el banner habitual SSH-2.0-OpenSSH_{9,0p1Ubuntu-1ubuntu7,3}. Cliente envía *SSH-2,0-OpenSSH_{8,3p1Ubuntu-1ubuntu0,1}*.

1.8.4. C4/S1

El cliente y el servidor son el mismo contenedor (loopback), el handshake TCP ocurre sobre 127.0.0.1.

El cliente envía *SSH-2.0-OpenSSH_{9,0p1Ubuntu-1ubuntu7,3}*, idéntico al servidor.

El Key Exchange Init del cliente es grande, luego el servidor envía su KEX Init. Se realiza ECDH Key Exchange, negociación de nuevas llaves, y los paquetes finales son cifrados localmente.

En este caso no hubo tráfico externo, solo loopback, por eso el flujo es más rápido y limpio.

1.9. Diferencia entre C1 y C2

La diferencia principal está en la versión del cliente SSH: C1 usa OpenSSH 7.3p1, C2 usa 7.7p1. Esto hace que los tamaños de los paquetes de Key Exchange Init sean distintos y que los algoritmos listados puedan variar. El patrón de tráfico y las huellas digitales (HASSH) serían diferentes.

1.10. Diferencia entre C2 y C3

C3 tiene un cliente más moderno: OpenSSH 8.3p1. El tamaño del Key Exchange Init es mayor que C2, y nuevamente los algoritmos y la disposición de los mismos pueden cambiar. La diferencia de flujo es visible en el tamaño de los paquetes y en la forma en que se negocian las nuevas llaves.

1.11. Diferencia entre C3 y C4

C4 es tanto cliente como servidor, sobre localhost. El cliente es OpenSSH 9.0p1, igual que el servidor, por lo que el Key Exchange Init tiene un tamaño diferente a C3 y el tráfico es más directo (no atraviesa la red). El patrón de paquetes es más rápido y sin retrasos de red.

2. Desarrollo (Parte 2)

2.1. Identificación del cliente SSH con versión “?”

Observando el intercambio de paquetes realizado entre cliente y servidor en los casos C1-S1, C2-S1, C3-S1 y C4-S1, se puede afirmar que la version del cliente utilizada es OpenSSH 7.3p1, como se observa en sus respectivas imagenes en "Client: Protocol (SSH-2.0-OpenSSH-7.3p1 Ubuntu-1ubuntu0.1)"

2.2. Replicación de tráfico al servidor (paso por paso)

Para replicar el tráfico SSH observado en los pcaps de los clientes C1, C2 y C3 hacia el servidor C4/S1, se realizó una secuencia controlada de pasos que reproduce el flujo mostrado en la figura de referencia: handshake TCP, envío de banner SSH, intercambio de claves, y establecimiento de sesión cifrada. A continuación se describe el procedimiento general utilizado.

2.2.1. Preparación del entorno

1. Se inicia el contenedor del cliente correspondiente (C1, C2 o C3).
2. Se confirma que el servidor SSH (sshd) esté ejecutándose dentro del contenedor C4/S1.
3. Se obtiene la dirección IP interna del servidor mediante: `docker inspect -f 'range .NetworkSettings.Networks.IPAddressend' C4`

2.2.2. Inicio de captura de paquetes

Desde el host, se inicia una captura dentro del contenedor cliente usando tcpdump, especificando la interfaz de red y el puerto 22 para capturar únicamente tráfico SSH. Ejemplo (para C1): `docker exec -d C1 bash -c "tcpdump -i eth0 -s 0 -w /tmp/c1_replicado.pcap tcpport22"`

Esta captura se mantiene en segundo plano mientras se realiza la conexión.

2.2.3. Generación del tráfico hacia el servidor

Se ejecuta una conexión SSH simple desde el cliente hacia el servidor utilizando autenticación por contraseña para reproducir el mismo flujo observado en el pcap base. Por ejemplo: `docker exec -it C1 ssh -oStrictHostKeyChecking=no prueba@IP_C4 > true`

El comando true se utiliza para crear una sesión mínima sin ejecutar comandos interactivos, lo que permite replicar exactamente los paquetes del handshake y el intercambio de claves.

2.2.4. Finalización de la captura

Una vez completada la conexión, se detiene la captura de paquetes dentro del contenedor mediante: `docker exec C1 pkill tcpdump`

2.2.5. Extracción del archivo pcap generado

El archivo pcap es copiado desde el contenedor al sistema anfitrión para posteriormente analizarlo con Wireshark o TShark.

```
docker cp C1:/tmp/c1_replicado.pcap./c1_replicado.pcap
```

2.2.6. Análisis del flujo replicado

El archivo pcap obtenido contiene el mismo patrón general observado en la figura de referencia, lo cual fue verificado:

1. Paquetes TCP iniciales: SYN, SYN/ACK, ACK.
2. Banner SSH enviado por el servidor.
3. Banner SSH enviado por el cliente.
4. Paquete SSH MSG KEXINIT del cliente (tamaño dependiente de la versión).
5. Intercambio ECDH.
6. Mensajes “New Keys”.
7. Primeros paquetes cifrados de la sesión.

Aunque los números exactos de secuencia pueden variar, el flujo y los tamaños generales coinciden con el patrón de tráfico esperado.

2.2.7. Repetición para C2, C3 y C4/S1

El mismo procedimiento se repitió para C2 y C3 con el objetivo de replicar el tráfico y obtener un pcap válido para cada cliente. En el caso de C4 actuando como cliente (loopback), la captura se realizó sobre la interfaz lo dentro del mismo contenedor, replicando también el tráfico local generado.

3. Desarrollo (Parte 3)

3.1. Replicación del KEI con tamaño menor a 300 bytes (paso por paso)

4. Desarrollo (Parte 4)

4.1. Explicación OpenSSH en general

OpenSSH (Open Secure Shell) es un conjunto de herramientas que implementa el protocolo SSH (Secure Shell) para proporcionar comunicaciones seguras sobre redes inseguras. Su

principal objetivo es permitir la autenticación remota y la transferencia segura de datos entre clientes y servidores, protegiendo la información mediante cifrado, autenticación de usuarios y verificación de integridad.

OpenSSH incluye varias utilidades, entre las que destacan:

1. ssh: cliente que inicia sesiones seguras hacia servidores SSH.
2. sshd: demonio que corre en el servidor y atiende conexiones entrantes.
3. scp y sftp: herramientas para transferencia segura de archivos.
4. ssh-keygen: generación de claves públicas y privadas para autenticación sin contraseña.

El protocolo SSH soporta cifrado simétrico, asimétrico y algoritmos de hashing para asegurar confidencialidad, autenticidad y detección de modificaciones en los datos.

4.2. Capas de Seguridad en OpenSSH

OpenSSH protege las comunicaciones en varias capas:

Capa de Transporte (Transport Layer Security):

1. Establece un canal seguro entre cliente y servidor.
2. Realiza intercambio de claves (Diffie-Hellman, ECDH) para generar una clave simétrica temporal.
3. Cifra toda la comunicación usando algoritmos como AES, ChaCha20, 3DES.
4. Verifica la integridad de los datos mediante HMAC o SHA.

Capa de Autenticación:

1. Permite identificar usuarios mediante contraseñas, llaves públicas/privadas o autenticación multifactor.
2. Soporta autenticación basada en certificados y agentes de claves (ssh-agent).

Capa de Conexión (Connection Layer):

1. Multiplexa canales dentro de la misma sesión SSH.
2. Cada canal puede ser terminal interactivo, reenvío de puertos o transferencia de archivos.
3. Garantiza confidencialidad y autenticidad de cada subcanal.

Seguridad adicional:

1. Protege contra ataques de repetición mediante números de secuencia y salting en HMAC.
2. Previene ataques de tipo Man-in-the-Middle (MITM) verificando la clave del host.

4.3. Identificación de que protocolos no se cumplen

Durante las pruebas realizadas con los contenedores C1, C2, C3 y C4 se observaron ciertos incumplimientos o desviaciones respecto a las mejores prácticas de seguridad SSH:

1. Autenticación de contraseña habilitada:

Los clientes se conectaron usando usuario y contraseña (prueba/prueba).

La autenticación basada en contraseña es más vulnerable a ataques de fuerza bruta en comparación con el uso exclusivo de claves públicas/privadas.

2. Versiones antiguas de OpenSSH en clientes

C1: OpenSSH 7.3p1

C2: OpenSSH 7.7p1

C3: OpenSSH 8.3p1

Versiones antiguas pueden carecer de correcciones recientes de seguridad y soportan algoritmos más débiles (ej. 3DES o CBC).

3. Ausencia de verificación HASSH en el entorno

No se pudo capturar ni comparar HASSH para identificar clientes de manera única.

La falta de esta verificación impide la detección automática de patrones de clientes y posibles intrusiones.

4. Tráfico de loopback en C4/S1

Al realizar conexiones locales, se observa que no se aplica control de políticas de red externas, lo que podría ocultar vulnerabilidades si no se analiza la configuración local del servidor.

Conclusiones y comentarios

1. Diferencias de versiones de OpenSSH y su impacto: Se observó que cada cliente (C1, C2, C3 y C4) utiliza distintas versiones de OpenSSH, lo cual afecta directamente el tamaño de los paquetes Key Exchange Init (KEXINIT) y la lista de algoritmos soportados. Clientes más modernos (C3 y C4) envían KEXINIT más largos, indicando soporte para más algoritmos o nombres de algoritmos más largos, mientras que versiones antiguas (C1, C2) tienen listas más cortas y, por lo tanto, patrones de tráfico diferentes. Esta diferencia se refleja también en las huellas digitales HASSH.
2. Flujo de comunicación SSH: El análisis de los pcaps muestra que todos los clientes respetan el flujo básico de SSH: handshake TCP, intercambio de banners, KEXINIT, ECDH, establecimiento de nuevas llaves y paquetes cifrados. Sin embargo, la secuencia exacta y los tamaños de los paquetes varían según la versión del cliente y la configuración

de la sesión. El loopback en C4/S1 demuestra un tráfico más rápido y limpio, al no atravesar la red externa.

3. Seguridad y autenticación: Durante las pruebas se identificó que todos los clientes utilizaron autenticación por contraseña, lo que no es recomendado para entornos productivos debido a su vulnerabilidad frente a ataques de fuerza bruta. Se recomienda migrar a autenticación mediante llaves públicas y deshabilitar contraseñas cuando sea posible.

4. Cumplimiento de buenas prácticas:

Los clientes y el servidor cumplen con el cifrado simétrico, asimétrico y verificación de integridad mediante HMAC.

No se cumple la verificación de HASSH para identificar clientes de manera única.

Versiones antiguas de OpenSSH (C1, C2) podrían carecer de parches de seguridad y soportar algoritmos obsoletos, lo que debe considerarse para entornos de producción.

La prueba de loopback (C4/S1) permite verificar la funcionalidad de SSH localmente, pero no reemplaza pruebas sobre la red para evaluar vulnerabilidades externas.

5. Comentarios generales: El estudio permitió comprender en detalle el flujo de paquetes SSH y cómo las versiones de OpenSSH influyen en la huella digital de la sesión. Además, la replicación de tráfico mediante contenedores Docker demostró ser una herramienta útil para analizar y validar comportamientos sin comprometer entornos de producción. Se recomienda mantener siempre actualizadas las versiones de OpenSSH y revisar periódicamente las políticas de autenticación y los algoritmos soportados, para garantizar comunicaciones seguras.